

DoxGPT Cost-Estimate Pipeline — AI Reference

For AI assistants helping solo practitioners adapt the insurance-verification and patient-cost-estimate workflow to their own EMR and clearinghouse stack.

System Context

This pipeline turns a day's scheduled-patient roster into per-patient cost estimates in three phases:

1. **Extract** — pull patient demographics from the EMR into one CSV per payer, in the format the clearinghouse expects for batch eligibility verification.
2. **Verify** — upload each per-payer CSV to the clearinghouse, run batch eligibility checks, download structured response CSVs.
3. **Estimate** — feed each response row into a HIPAA-covered LLM (DoxGPT) with a prompt that combines the practice's fee schedule, planned-procedure bundle, and calculation rules to produce a cost estimate.

The reference implementation uses **ModMed EMA** (Angular) as the EMR, **Availity Essentials** as the clearinghouse, and **DoxGPT** as the LLM. All three are HIPAA-covered with signed BAAs.

HIPAA Constraint

PHI moves through three vendors. Each must hold a BAA covering the practice's use. The browser-side extraction script in Phase 1 runs locally and transmits nothing — that is intentional and non-negotiable for the architecture. The CSV(s) are written to the user's local Downloads folder. The first PHI transmission outside the user's machine is the Availity upload.

Pasting Availity output into a generic ChatGPT, Claude, Gemini, or Grok account breaks the chain. The prompt in Phase 3 is designed for DoxGPT specifically because Doximity holds a BAA covering individual physician use.

Key DOM Selectors (ModMed Eligibility Report)

The Eligibility Report view is reached from **Appt Flow** → **View Eligibility Report** (badge link). It is a different table component than the Tasks / Referrals views — standard HTML  with a stable ID, not PrimeNG.

Element	Selector
Data table	<code>#reportTable</code>
Header cells	<code>#reportTable thead th, #reportTable thead td</code>
Body rows	<code>#reportTable tbody tr</code>

Column headers as of this writing: Patient , Patient Phone , Appointment , Payer , Policy Level , Policy Number , Status , Review Status , Action . The script matches on lowercase exact match against patient , payer , and policy number .

Column-Mapping Strategy

The script does not hard-code column indices. Instead it reads the header row, normalizes header text, and matches on exact lowercase string:

- `patient` → column containing patient block (name + DOB + MRN + PMS ID stacked)
- `payer` → column with payer name (e.g. "Medicare of Texas")
- `policy number` → column with member ID (the Availity-bound "member_id")

The Patient column is a stacked block of multiple data points. The script handles it by:

1. Splitting on newline; the first line is the patient name in Last, First M format.
2. Regex-matching for MM/DD/YYYY anywhere in the cell text to extract DOB without depending on a specific separator.

This approach is resilient to whatever Angular renders the cell as (block elements, line breaks, multiple spans) because it does not depend on a particular HTML structure — just the presence of a name line and a date pattern.

Multi-Payer Split

Availity's batch endpoint runs against one payer's roster per submission. The script groups rows by payer name (as it appears in the Payer column), generates one CSV per payer, and triggers a download per file. Chrome throttles multi-download attempts from a single user action; the script staggers downloads by 300ms to bypass this. The first multi-payer run also requires the user to grant Chrome's "Allow multiple downloads" permission for the ModMed origin.

Filename pattern: `availability_.csv` . Slug is lowercase with non-alphanumeric chars collapsed to underscores.

Critical Implementation Notes

Patient cell parsing: The Patient cell contains name, DOB, MRN, and PMS ID on separate lines. Use `innerText.split(/\n\r/)[0]` for the name line and `innerText.match(/\\d{1,2}\\/\\d{1,2}\\/\\d{4}/)` for DOB. Do NOT split on " / " literally — the cell uses newlines, not visible slashes.

Name parsing: ModMed renders names as Last, First M in the Patient cell's first line. Split on the first comma — not on whitespace, because some last names contain spaces (e.g. "Van Buren").

Empty rows: Skip rows where `cells.length < 2` to handle any placeholder rows.

CSV escaping: Wrap any field containing a comma or quote in double quotes and double any internal quotes. Names like O'Brien, Mary and free-text payer names like BCBS, BlueChoice PPO will hit this.

Availity batch CSV column order: Availity is strict about column order on intake. The canonical Availity Essentials batch eligibility header is: last_name, first_name, dob, member_id, payer, provider_npi. Other clearinghouses (Office Ally, Waystar, Change Healthcare relay) use different headers — check the documentation for the specific batch endpoint.

Availity response — Pending rows: Availity returns Pending when the payer has not responded yet. Pending rows have null values for deductible, OOP, copay, coinsurance. The DoxGPT prompt is built to recognize null fields and refuse to fabricate a number — it will flag the patient for re-check instead. Do not pre-fill nulls with zeros.

Availity response — codes vs. text: Some Availity configurations return enumeration values as text (Active Coverage, Inactive, Pending) and some return them as codes (AC, IN, PE). The DoxGPT prompt is written for text. If your Availity returns codes, translate before pasting into DoxGPT.

Payer name normalization: Payer names as they appear in ModMed (e.g. "Medicare of Texas") may not match Availity's internal payer list verbatim. Availity will reject unrecognized payer strings on upload. The user fixes mismatches in the CSV before re-upload. This is a known integration friction and not something the extraction script can pre-solve.

DoxGPT Prompt Architecture

The prompt has five structural elements that matter more than they appear:

1. **Fee schedule in the prompt body** — not in DoxGPT's memory. Every run uses the exact schedule pasted. No drift between what the model "knows" and what the practice's contracts actually say.
2. **Insurance fallback for uncontracted payers** — prevents hallucinated allowables. Falling back to the most conservative column protects against under-collection.
3. **Worst-case planned bundle with refund posture** — single number out, refund what is not performed. Avoids under-collection on visits that grow in scope.
4. **Mutually-exclusive procedures clause** — prevents double-counting CPTs that cannot both bill at the same visit. Without this clause, estimates inflate by the price of the cheaper of any conflicting pair.

5. **Flag rules for Auth Required and Inactive coverage** — the model is instructed to flag for staff review rather than produce a confident dollar amount when the data does not support one. This is the difference between a usable cost estimator and a hallucinating one.

When adapting the prompt to a different specialty, the practice changes the fee schedule and the planned bundle. The five structural elements above stay as written.

Workflow Sequence

1. User opens **Appt Flow** → **View Eligibility Report**, sets date filter and pagination to "show all," opens Chrome DevTools console.
2. User pastes extraction script. Script reads the visible `#reportTable`, groups rows by payer, writes one `availability_.csv` per group locally. First run prompts Chrome's multi-download permission.
3. User uploads each per-payer CSV to the clearinghouse batch eligibility endpoint, waits 3-5 minutes per batch (parallelizable), downloads each response CSV.
4. User opens DoxGPT, pastes prompt template (customized once for their practice), then pastes Availability response rows.
5. DoxGPT returns structured cost estimates per patient.

Adapting This Pattern to a Different Stack

If your stack is not ModMed + Availability + DoxGPT, paste this entire reference into Claude, Grok, or another modern AI along with three pieces of information:

1. **The name of your EMR** (e.g. Athenahealth, eClinicalWorks, Epic Hyperspace, Practice Fusion, DrChrono) and what its scheduled-patients or eligibility-report page looks like in the browser. The AI will need to identify the equivalent DOM selectors. For a quick start, right-click any row in that view, choose Inspect, and copy the outer HTML of one row into the chat.
2. **The name of your clearinghouse** (e.g. Office Ally, Waystar, Change Healthcare, pVerify, Trizetto) and whether it supports batch eligibility upload. If it does, the AI needs your clearinghouse's batch CSV header format — copy it from their documentation. Note whether the clearinghouse expects one-payer-per-batch (like Availability) or mixed-payer batches; this changes whether the script needs the multi-payer split.
3. **Which HIPAA-covered LLM you use.** DoxGPT (Doximity) is the most common for individual physicians because the BAA covers personal use. Other covered options include Google Vertex AI with a signed BAA, Anthropic Claude via Bedrock with a BAA, or OpenAI's Enterprise tier with an executed BAA. Generic personal accounts on those same services are NOT covered.

Ask the AI to adapt the same three-phase architecture — extract from EMR with per-payer split, batch-verify through clearinghouse one payer at a time, estimate via HIPAA-covered LLM — to your specific stack. The AI now has a working reference implementation to mimic rather than inventing the workflow from scratch.

The architecture is the unlock. The specific code is the easy part once the architecture is clear.

Companion to: Cost-Estimate Tutorial (this bundle). Course module: Module 5 — Insurance Verification & Patient Cost Estimates.